

Day 8

UART Receiver:

Verilog Code:

```
module uart_rx #(
    parameter CLK_FREQ = 50_000_000,
    parameter BAUD_RATE = 9600
)(
    input    clk, rst,
    input    rx,
    output reg [7:0] rx_data,
    output reg    rx_done, // pulses high for 1 clk when byte received
    output reg    rx_busy
);
    localparam BAUD_DIV  = CLK_FREQ / BAUD_RATE;
    localparam HALF_BAUD = BAUD_DIV / 2; // for start bit centering

    localparam IDLE = 2'b00;
    localparam START = 2'b01;
    localparam DATA = 2'b10;
    localparam STOP = 2'b11;

    reg [1:0] state;
    reg [12:0] baud_cnt;
    reg [2:0] bit_cnt;
    reg [7:0] shift_reg;

    // Synchronise rx input — 2-FF synchroniser for metastability
    reg rx_sync1, rx_sync2;
    always @(posedge clk) begin
        rx_sync1 <= rx;
        rx_sync2 <= rx_sync1;
    end
    wire rx_s = rx_sync2; // use this, not raw rx

    always @(posedge clk) begin
        if(rst) begin
            state <= IDLE;
            baud_cnt <= 0;
            bit_cnt <= 0;
            shift_reg <= 0;
            rx_data <= 0;
            rx_done <= 0;
            rx_busy <= 0;
        end
        else begin
            rx_done <= 0; // default — only high for 1 cycle

            case(state)
                IDLE: begin
                    rx_busy <= 0;
                    baud_cnt <= 0;
                    // Detect falling edge = start bit
                    if(rx_s == 1'b0) begin
                        state <= START;
                        rx_busy <= 1;
                    end
                end
            endcase
        end
    end
```

```

        end
    end

START: begin
    // Wait half bit period — lands us in middle of start bit
    // Then verify it's still low (not a glitch)
    if(baud_cnt == HALF_BAUD - 1) begin
        baud_cnt <= 0;
        if(rx_s == 1'b0) // confirmed start bit
            state <= DATA;
        else
            state <= IDLE; // was a glitch, go back
        end
    end
    else baud_cnt <= baud_cnt + 1;
end

DATA: begin
    if(baud_cnt == BAUD_DIV - 1) begin
        baud_cnt <= 0;
        // Sample at middle of each bit
        shift_reg <= {rx_s, shift_reg[7:1]}; // MSB comes in, shift right
        if(bit_cnt == 7) begin
            bit_cnt <= 0;
            state <= STOP;
        end
        else bit_cnt <= bit_cnt + 1;
    end
    else baud_cnt <= baud_cnt + 1;
end

STOP: begin
    if(baud_cnt == BAUD_DIV - 1) begin
        baud_cnt <= 0;
        if(rx_s == 1'b1) begin // valid stop bit
            rx_data <= shift_reg;
            rx_done <= 1'b1;
        end
        // if stop bit is 0 — framing error, discard silently
        state <= IDLE;
    end
    else baud_cnt <= baud_cnt + 1;
end
endcase
end
end
endmodule

```

UART Top module:

```

module uart_top #(
    parameter CLK_FREQ = 50_000_000,
    parameter BAUD_RATE = 9600
)(
    input    clk, rst,
    input    tx_start,
    input [7:0] tx_data,
    output [7:0] rx_data,
    output    rx_done,

```

```

output    tx_busy,
output    rx_busy,
output    tx_wire // expose for waveform viewing
);
wire tx_to_rx;

uart_tx #(CLK_FREQ, BAUD_RATE) u_tx(
    .clk(clk), .rst(rst),
    .tx_start(tx_start),
    .tx_data(tx_data),
    .tx(tx_to_rx),
    .tx_busy(tx_busy)
);

uart_rx #(CLK_FREQ, BAUD_RATE) u_rx(
    .clk(clk), .rst(rst),
    .rx(tx_to_rx),
    .rx_data(rx_data),
    .rx_done(rx_done),
    .rx_busy(rx_busy)
);

assign tx_wire = tx_to_rx;
endmodule

```

Testbench:

```

`timescale 1ns/1ps
module tb;
    parameter CLK_FREQ = 1_000_000;
    parameter BAUD_RATE = 100_000;

    reg    clk, rst;
    reg    tx_start;
    reg [7:0] tx_data;
    wire [7:0] rx_data;
    wire    rx_done, tx_busy, rx_busy, tx_wire;

    uart_top #(CLK_FREQ, BAUD_RATE) uut(
        .clk(clk), .rst(rst),
        .tx_start(tx_start),
        .tx_data(tx_data),
        .rx_data(rx_data),
        .rx_done(rx_done),
        .tx_busy(tx_busy),
        .rx_busy(rx_busy),
        .tx_wire(tx_wire)
    );

    initial begin clk = 0; forever #10 clk = ~clk; end

    initial begin
        $dumpfile("uart_top.vcd");
        $dumpvars(0, tb);
    end

    // Task — send byte and self-check
    task send_and_check(input [7:0] data);

```

```

begin
    // Wait until not busy
    wait(!tx_busy);
    @(negedge clk);
    tx_start = 1;
    tx_data = data;
    @(negedge clk);
    tx_start = 0;

    // Wait for RX to finish
    wait(rx_done);
    @(posedge clk);

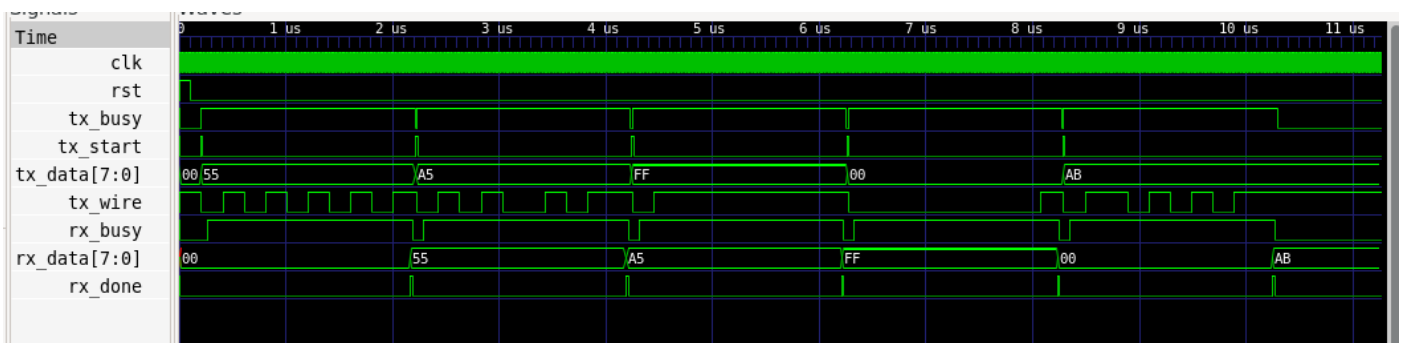
    // Self check
    if(rx_data == data)
        $display("PASS: sent %h received %h", data, rx_data);
    else
        $display("FAIL: sent %h received %h", data, rx_data);
end
endtask

initial begin
    rst = 1; tx_start = 0; tx_data = 0;
    #100; rst = 0;
    #100;

    send_and_check(8'h55);
    send_and_check(8'hA5);
    send_and_check(8'hFF);
    send_and_check(8'h00);
    send_and_check(8'hAB);

    #1000; $finish;
end
endmodule

```



Simulation and Synthesis output:

VCD info: dumpfile uart_top.vcd opened for output.

PASS: sent 55 received 55

PASS: sent a5 received a5

PASS: sent ff received ff

PASS: sent 00 received 00

PASS: sent ab received ab

uart_tbt.v:64: \$finish called at 11270000 (1ps)

=== \$paramod\$b7294f1997723f0618629b64f4b039e150234139\uart_tx ===

Number of wires:	47
Number of wire bits:	278
Number of public wires:	9
Number of public wire bits:	41
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	24
\$add	2
\$dff	3
\$eq	2
\$logic_not	1
\$mux	15
\$reduce_bool	1

=== \$paramod\$b7294f1997723f0618629b64f4b039e150234139\uart_rx ===

Number of wires:	101
Number of wire bits:	408
Number of public wires:	13
Number of public wire bits:	42
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	63
\$add	4
\$dff	9
\$eq	14
\$logic_not	3
\$mux	29
\$not	2
\$pmux	2

=== uart_top ===

Number of wires:	10
Number of wire bits:	24
Number of public wires:	10
Number of public wire bits:	24
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	2
\$paramod\$b7294f1997723f0618629b64f4b039e150234139\uart_rx	1
\$paramod\$b7294f1997723f0618629b64f4b039e150234139\uart_tx	1

=== design hierarchy ===

uart_top	1
\$paramod\$b7294f1997723f0618629b64f4b039e150234139\uart_rx	1
\$paramod\$b7294f1997723f0618629b64f4b039e150234139\uart_tx	1

Number of wires:	158
Number of wire bits:	710
Number of public wires:	32

Number of public wire bits:	107
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	87
\$add	6
\$dff	12
\$eq	16
\$logic_not	4
\$mux	44
\$not	2
\$pmux	2
\$reduce_bool	1